

Практика 01 · Hidden Gift 2.0

ROS 2 graph, topics, and a first action

90 минут · ROS 2 Jazzy · Python · индивидуальная работа · bonus не обязателен

1. Что должно получиться

Вы завершите основную логику action server `shad_interfaces/action/ExploreZone` для **одного корректного goal**. Робот обследует заданную область, возвращает центр зелёного прямоугольника либо подтверждает его отсутствие, отправляет простой feedback и явно останавливается перед завершением.

Проверяется поведение, а не совпадение с эталонным алгоритмом. LLM разрешены: результат их работы необходимо проверить теми же тестами. Алгоритм поиска не главная тема занятия, поэтому геометрические помощники уже даны.

Уровень	Что входит
Обязательн о	Окружение и ROS graph; один корректный goal; найденный подарок; подтверждённое отсутствие; простой feedback; явный zero <code>Twist</code> перед завершением
Расширени е	Отмена и старт внутри подарка
Позже в курсе	Одновременные goals/busy, отзывчивость health service, потеря датчика и deadline, произвольные namespaces, QoS-совместимость, callback groups и locks

Полный балл за L01 получается только из обязательной части. Bonus-cases не являются скрытыми требованиями.

Откуда взято задание и что изменено

Основа — предоставленное домашнее задание «Знакомство с ROS2»: прямоугольный подарок, управление `Twist`, датчик цвета и центр с точностью 0.05 м. Публичный kit не копирует legacy interface: для курса создан собственный `shad_interfaces/action/ExploreZone` с прямыми границами зоны и нейтральными полями результата.

В исходных PDF нет исходников `turtlesim_contest_supplements.tar.xz`. Поэтому здесь реализован **новый учебный headless-симулятор**, а не восстановлена старая модификация turtlesim. Он использует настоящие ROS 2 nodes, DDS, actions и сообщения `turtlesim/Pose`, `turtlesim/Color`, но рисуется в браузере. Gazebo появится в следующих занятиях.

В новом стенде сохранены ограниченные скорости, синхронизированное наблюдение цвета и позы, простой feedback и обязательный явный stop. Более сложные failure/concurrency cases остаются в инфраструктуре курса, но вынесены из обязательной части первого задания.

2. Подготовка — 20–30 минут до занятия

Достаточно Docker, Compose и Git. Linux, Mac Apple Silicon и Windows через WSL2 подходят для запланированного multi-arch образа; конкретный образ преподаватель должен проверить на amd64 и arm64 до выдачи. Локальные ROS, Gazebo, XQuartz и GPU не нужны.

Важно: преподаватель заранее публикует образ, фиксирует digest в `.env` и выдаёт адрес вашего репозитория. Не запускайте сборку образа во время занятия.

В терминале **хоста**, начиная из корня личного репозитория:

```
git switch master
cd task01
docker compose up -d --wait
./course doctor
```

Откройте IDE `http://127.0.0.1:8080` и поле `http://127.0.0.1:8081`. Встроенный терминал IDE уже знает ROS underlay. `./course doctor` на хосте вызывает `course doctor` внутри и передаёт результат проверки Docker. Вызванный только из IDE doctor честно помечает host Docker как `not_checked`: Docker socket внутрь не передаётся.

Doctor проверяет доступность Docker/Compose через хостовый wrapper, доступные внутренние порты, запись в workspace, видимое свободное место, ROS imports и **реальный headless smoke test**: discovery, `shad_interfaces/msg/SensorSample` и движение по `cmd_vel`. Отчёт сохраняется в `reports/environment.json`. Он не содержит hostname, токенов, SSH-ключей или полного окружения и должен попасть в итоговый коммит.

Если преподаватель просит заранее подтвердить готовность окружения, сделайте промежуточный commit из `task01/`. Он не заменяет итоговую сдачу:

```
git add reports/environment.json
git commit -m "L01: verify environment"
git push origin master
```

Если свободно меньше 3 GiB в видимом workspace, doctor завершится ошибкой. Рекомендация 30 GB относится к диску хоста в целом: образ, Docker cache и будущие занятия. Это не измеренный размер данного образа.

3. Быстрая проверка ROS graph — 0–10 минут

В терминале IDE:

```
ros2 node list
ros2 topic list -t
ros2 topic info /lab/sample --verbose
ros2 topic echo /lab/sample --once --qos-reliability best_effort
```

Должен быть узел `/lab/gift_world`. Сервер `/lab/gift_server` появится после `course build` и `course run`.

```
course build
course run
```

В другом терминале:

```
ros2 action list -t
ros2 interface show shad_interfaces/action/ExploreZone
ros2 action info /lab/explore_zone
```

Флаг QoS в готовой команде нужен для чтения sensor stream; разбор совместимости QoS будет в Week 2.

Вопрос проверки: action виден в графе. Доказывает ли это, что goal найдёт подарок, корректно подтвердит отсутствие и остановит робота? Нет: discovery подтверждает существование endpoint, а не выполнение behavioral contract.

4. Обязательный контракт

В обязательных тестах приходит один корректный goal. Начальная позиция находится вне подарка; случай старта внутри него оставлен как bonus.

Объект	Требование
Мир	Квадрат 11×11 м; робот — точка с unicycle-кинематикой
Подарок	Один зелёный прямоугольник со сторонами по осям, каждая сторона ≥ 1 м; либо подарка в области нет
Search area	Корректный прямоугольник внутри мира; валидация некорректных координат уже дана в adapter
Управление	Только cmd_vel, geometry_msgs/Twist; teleport и simulator internals запрещены
Ограничения	$\text{abs}(\text{linear.x}) \leq 2$, $\text{abs}(\text{angular.z}) \leq 3$; NaN/Inf запрещены
Action	Один goal к explore_zone; goal содержит min_x, min_y, max_x, max_y
Feedback	Во время работы приходит простой feedback о ходе поиска; точная частота и метрика пути не оцениваются
Найдено	Вернуть target_found=true и target_x, target_y с евклидовой ошибкой ≤ 0.05 м
Не найдено	Только после полного покрытия вернуть target_found=false
Завершение	Оба штатных исхода — SUCCEEDED; перед result сервер явно публикует zero Twist

В browser debug-view можно показать подарок. **Решению запрещено читать HTTP /state, исходные данные сцены или внутренности симулятора.** Во внешней проверке такого HTTP-сервера и этих файлов у студента нет.

Bonus и последующие недели

Расширение L01: старт внутри подарка и корректный terminal status при отмене. Эти случаи можно делать после обязательных `found` и `absent`. Отклонение некорректной области уже реализовано в предоставленном adapter и служит примером входной валидации, а не отдельной работой студента.

Не входят в L01 core: второй одновременный goal, health service под нагрузкой, sensor-loss/deadline policy, namespace/remapping, отдельный QoS-эксперимент, callback groups, locks и анализ гонок. Эти темы остаются в лекционных демонстрациях и возвращаются в следующих неделях.

5. Action result: два обязательных исхода

Ситуация	Terminal status	target_found
Найден центр	SUCCEEDED (4)	true
Область полностью проверена, подарка нет	SUCCEEDED (4)	false

Отсутствие — успешный результат полного поиска, а не ошибка. Для расширения с отменой ожидаются `CANCELED (5)` и `target_found=false`; некорректный goal отклоняется предоставленным adapter до выполнения.

6. Что реализовать — 10–55 минут

Поиск: `mission.py`

В классе `Mission` реализуйте конечный автомат. Не вызывайте ROS из этого файла.

`raster(area)` выдаёт маршрут покрытия. `drive_to(sample, target)` возвращает `(linear_velocity, angular_velocity, waypoint_reached)`. Когда обнаружен зелёный sample **внутри** границ goal, создайте `RectangleProbe(area, sample)` и передавайте ему последующие samples. Помощник уже умеет физически измерить границы прямоугольника. Он возвращает `Decision` и в конце `Outcome('found', x, y)` либо `Outcome('abort')`.

Если маршрут исчерпан без обнаружения — `Outcome('absent')`. Возвращайте простой feedback через `Decision`: обязательная проверка требует, чтобы сообщения приходили во время работы, но не оценивает целевую частоту, монотонный distance или сложную failure policy.

Навигационный помощник работает при явно заданных предположениях: точечный робот, прямоугольник по осям, идеальный датчик. Это не универсальный алгоритм восприятия реального робота.

Результат action: `server.py`

В обязательной части сопоставьте `Outcome('found')` и `Outcome('absent')` с успешным terminal status. Adapter уже содержит publishers, subscriptions, action server, feedback scheduling и явный stop перед result.

Student-facing TODO для core ограничены `Mission.step()` и отображением `found/absent` в `SUCCEEDED`. Политики `validation`, `cancellation`, `busy/concurrency`, `health responsiveness`, `sensor timeout`, `task deadline`, `namespaces` и `callback synchronization` предоставляются инфраструктурой и не оцениваются как работа студента в L01. После core можно запустить `bonus-cases`, чтобы проверить, как ваш `state machine` работает с готовым `adapter`.

7. Проверка по шагам — 55–80 минут

```
course build
course test --case found
course test --case absent
# То же самое одной командой:
course test
```

Эти два `cases` составляют обязательную функциональную проверку. Они также проверяют наличие `feedback` и явный `stop`. Команда `course test` запускает только этот core; расширенные `platform-regression cases` не входят в обычный `student run`.

После core можно попробовать `bonus`:

```
course test --case inside
course test --case cancel
```

Для ручной диагностики запустите `course run`, затем в другом терминале:

```
course goal --area 2 2 8 8
```

Наблюдайте `robot view`, `feedback` и `terminal status`. Остановка только за счёт встроенного `watchdog` не засчитывается: сервер должен явно отправить нулевой `Twist`.

Типичные симптомы

Симптом	Что проверить первым
Action отсутствует	<code>course build</code> , процесс <code>server</code> , <code>namespace</code> , <code>source overlay</code>
Goal принят, робот стоит	Есть ли <code>sample</code> ; не возвращает ли <code>Mission</code> всегда <code>zero Decision</code>
Пустая область завершилась <code>ABORTED</code>	Не заменили ли проверку покрытия общим <code>timeout</code>
Gift найден, но <code>result false</code>	Переход <code>Mission</code> в <code>found</code> и отображение <code>terminal result</code>
Test сообщает о движении после <code>result</code>	Явный <code>zero Twist</code> должен быть опубликован до завершения

8. Баллы: 100 за практику

Проверка	Баллы
<code>found</code> : найти подарок, вернуть центр с требуемой точностью, отправить <code>feedback</code> и явно остановиться	55
<code>absent</code> : подтвердить отсутствие только после покрытия, вернуть <code>SUCCEEDED</code> , отправить <code>feedback</code> и явно остановиться	45

Extension-cases `inside` и `cancel` отмечаются отдельно и не уменьшают `core-score`. `invalid`, `busy`, `feedback_health`, `sensor_loss`, `namespace` и `deadline` не входят в оценку L01 даже если остаются в `internal regression harness`.

Публичные тесты — обратная связь. Итоговый запуск использует преподавательский образ и другие `seeds`, но проверяет тот же опубликованный `core contract`. Изменения студентом тестов, `Dockerfile` или `workflow` не меняют критерии. Инфраструктурная ошибка должна идти на перепроверку, а не автоматически превращаться в ноль.

В масштабе курса: **90% — среднее лучших 9 из 10 практик; 10% — итоговый ролик до 3 минут.** Отдельной устной защиты и выбора треков нет.

9. Сдача — последние 10 минут

Заполните `docs/verification.md`: для `found` и `absent` укажите `PASS` или `FAIL` (при ошибке — одну короткую причину), затем одну конкретную границу решения и использовали ли вы LLM. Если `core-problem` не обнаружено, границей может быть `bonus inside` или `cancel`, который вы не реализовывали или не проверяли. Полные логи, Student ID и Commit SHA не нужны. Достаточно 5–10 строк собственных ответов.

`docs/verification.md` обязателен как краткая запись самопроверки, но отдельных баллов за него нет и авто-грейдер его не разбирает. Оценка определяется поведением решения в `found` и `absent`; этот файл не заменяет код и тесты.

Все файлы решения должны остаться внутри уже существующей папки `task01/`. Закоммитьте `src/hidden_gift/`, `reports/environment.json` и `docs/verification.md`, затем отправьте ветку `master`:

```
# Терминал хоста, из task01/:
git switch master
git status --short
git add src/hidden_gift reports/environment.json docs/verification.md
git diff --cached --stat
git commit -m "L01: implement and verify Hidden Gift action"
git push origin master
```

Не коммитьте `build/install/log` и не присылайте файл с самостоятельно написанной оценкой.

Commit SHA никуда вписывать и коммитить не нужно. SHA появляется только после создания коммита. В момент дедлайна преподаватель сам сохранит идентификатор последнего коммита ветки `master`, который находится на GitHub. От студента требуется только разместить код в

`task01/`, сделать commit и выполнить `git push origin master`. После push убедитесь на GitHub, что изменения появились.

10. Если окружение не запустилось

На Linux с UID не 1000 задайте `LOCAL_UID` и `LOCAL_GID` в `.env` по `id -u` / `id -g`. На Windows выполняйте команды из WSL2. При занятом порте измените `IDE_PORT` / `SIM_PORT` в `.env` и перезапустите Compose.

Для удалённого Linux используйте SSH tunnels, оставляя сервисы привязанными к localhost:

```
ssh -L 8080:127.0.0.1:8080 -L 8081:127.0.0.1:8081 USER@HOST
```

Не открывайте code-server без аутентификации в интернет. Для проблем установки преподаватель должен иметь проверенный remote fallback, а не требовать покупки нового ноутбука.

Официальные материалы

ROS actions: <https://design.ros2.org/articles/actions.html>

Callback groups: <https://docs.ros.org/en/jazzy/How-To-Guides/Using-callback-groups.html>

Python action tutorial: <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Writing-an-Action-Server-Client/Py.html>

Docker multi-platform: <https://docs.docker.com/build/building/multi-platform/>